

我眼中的BFF

引言

作为一个以BFF命名的团队，经常会有同学问及这些问题：

你是前端开发吗？你们是不是写NodeJS的？你们是无服务端吗，这个组织不是已经解散了吗？我们新上了一个很牛逼的FAAS平台，要不要试试？BFF不就是FAAS吗，集团那么多FAAS，你们还要再造个轮子？

我的解释通常是：近端侧的服务端，也就是字面理解的：为大前端服务的后端（Backend For Frontend）。这和传统意义的后端有什么不同？我的理解是离炮火更近，图1是年初给大前端同学普及服务端技术的开篇一页，BFF是把靠近端侧的部分又拆出了一层，基于性能的考虑实现方案可能会放在边缘，前后端边界线会下移，这里先忽略。

软件领域大部分问题都可以：

- 通过引入一个中间层解决

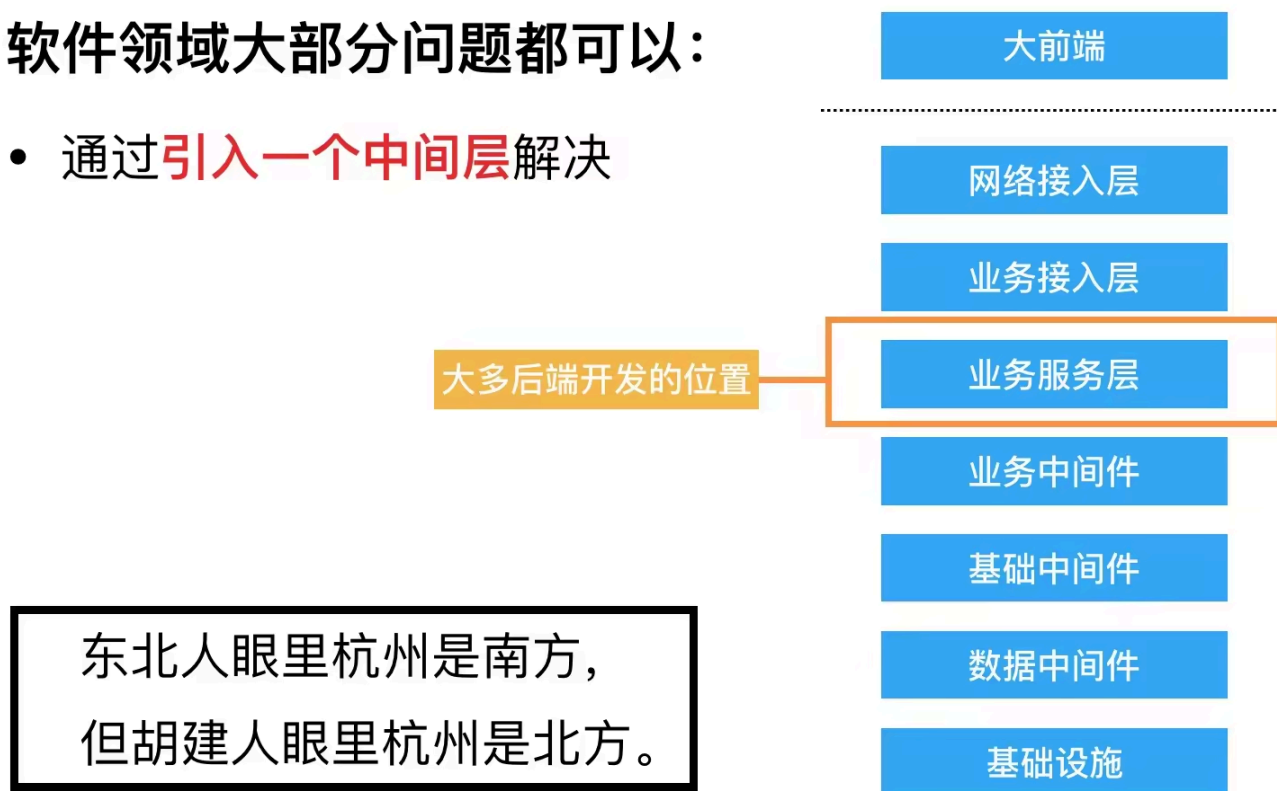


图1.给大前端同学的服务端技术分享

引入BFF层核心解决什么问题？是为了促进就业？提升研发效率？还是新瓶装老酒？

毛爷爷说：“问题就是事物的矛盾。哪里有没有解决的矛盾，哪里就有问题。”所以我尝试用这样的定义来描述BFF层的问题域：追求相对稳定的垂直业务域和灵活多变的端侧需求的矛

盾。可以简单类比为计算机领域无处不在的缓冲，来协调吞吐速度相差很大设备间的数据传送。除此之外还有其他一些背景价值，如下图2列出来。

Why BFF?

- **历史遗留业务支撑**（mobile web全业务，pc web交易，详情等）
Legacy BFF code after Sirius
- 相对**稳定的业务域与多变的端需求**之间的矛盾，中台能力如何快速投放多端，新业务App如何快速搭建
Gap between stable middle-platform domains and versatile requirement in terminals
- 补齐**端侧差异化投放能力**（国家，语种，人群，AB）
Delivery ability by different strategies, such as country, language, group, ab etc.
- 无中台对应的**横向聚合场景**（home，detail，车型库，运营静态配置，全站坑位等）或基于**聚合的优化**（会场多请求合并，端到端多级容灾）支撑
Horizontal scenarios, such as home, detail, my vehicle, operation configuration etc.
- 从**端出发的业务效能**评估（abtest、导购效能模型等）
Business effectiveness optimization from terminals

图2.微服务架构、中台化背景下BFF的价值

基于上面的问题域思考就会发现BFF的价值远远大于NodeJS的前后端技术栈统一、FAAS化的胶水代码、无线服务端的各种Adaptor。它不仅涵盖助力研发提效（率），更重要的包括助力业务提效（能），这里的效率指的是能快速上线，效能则对应上线的功能真正带来业务价值。

简介

BFF的概念源于2015年Sam Newman的博客[文章](#)，文章介绍了引入BFF的价值，探讨了几种BFF划分模式，基本是遵从康威定律，技术架构和组织架构相匹配，文章的副标题为“Single-purpose Edge Services for UIs and external parties”很能说明BFF更多是Edge Service，近端侧，如图3。

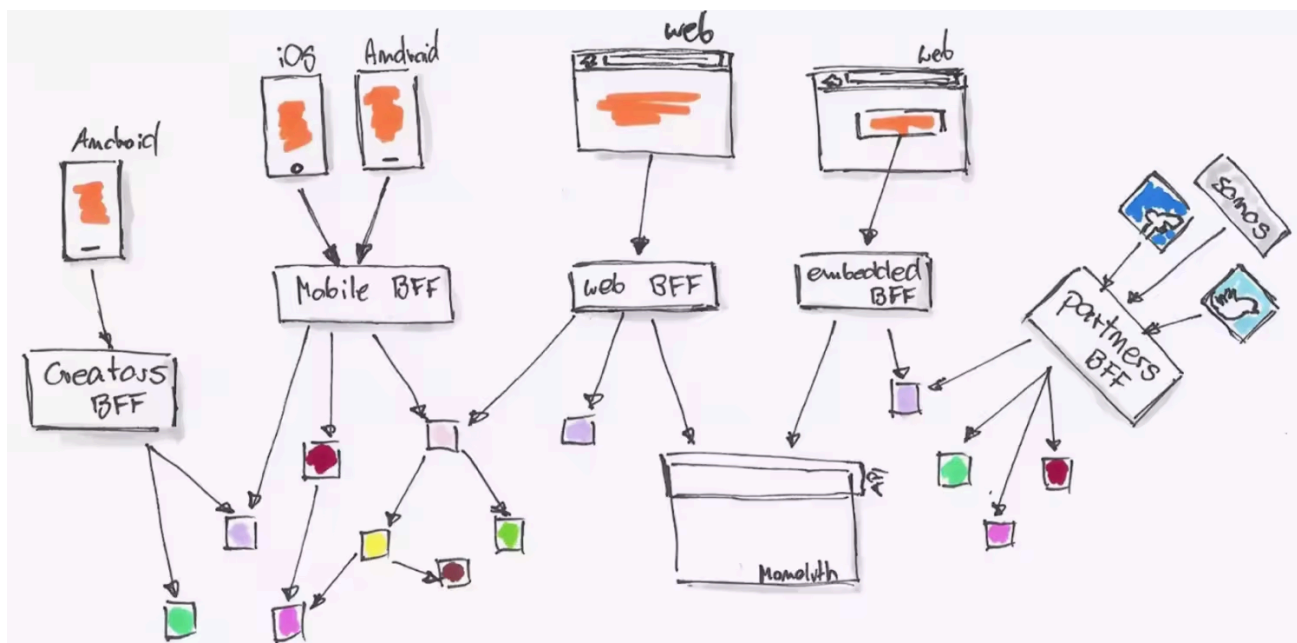


图3 近端侧的BFF架构

所以BFF技术更多在大前端圈流行，也就不难理解大多基于NodeJS技术栈的疑问。关于BFF在集团各个团队已经有诸多实践，ATA搜索有1000+结果，如下图4。



图4 ATA搜索BFF

BFF的“三化”

作为一个前无线服务端成员，对基于Java+Velocity构建的单体应用的臃肿低效，都2070年了还采取硬编码Adaptor，改个JS版本也要发Java应用的模式深恶痛绝。我想这也是诸多技术同学拥抱BFF，采取前后端分离，引入组件化的痛点之一。回到我眼中的BFF，理想的模式是什么？

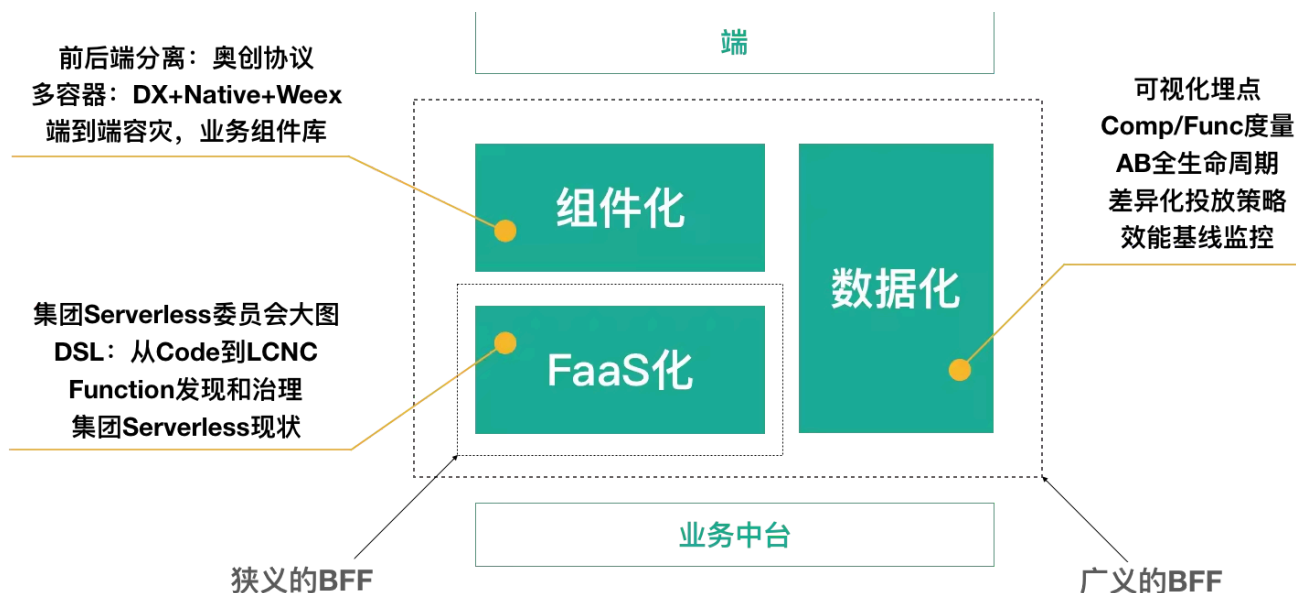


图5 BFF范畴的扩展

图5中我把BFF的范畴做了扩展，为什么这么做？我记得癡总表达过集团的基础中间件已经非常牛逼，但往上触及业务的解决方案还很初级（应该是类似的意思），而越贴近业务难做，但带来的增量价值越高。具体：

- **组件化**（Component-Oriented）：这里的组件不是简单的无业务含义的UI控件，而是具备业务含义，面向某个业务域沉淀的业务组件，例如国际化支付域的信用卡组件，它是一个自包含的，包括组件的数据采集规范，UI控件定义，开放给运营的配置项，默认渲染/校验/提交逻辑，及其扩展点。
- **FaaS**（Function as a Service）：这个都比较熟悉，一般理解是广义的FaaS，包括函数的生命周期管理和函数运行时，详细定义和规划可以参考集团Serverless委员会的[文章](#)。
- **数据化**（Data-Driven）：无论是组件层还是Function层，核心目标都是复用提效，数据驱动的目的就是对这两层价值做度量，并借助科学的方法为产品优化提供支撑。

组件化

软件领域的终极追求：**复用**！大家的思考路径都是类似，就像乐高积木一样基于标准件，快速组合，形成丰富多彩的世界。早前写过一篇对组件化的[思考](#)，粒度问题，自组织（健康度，无痕埋点，容灾，测试完备，规范治理等）都有讨论，这里不再赘述。

回到集团的组件化方案，百花齐放，**不是没得选，是选择太多**，目前比较主流的方案：

- 基础链路：稳定性，审核风险，主要是DinamicX，Flutter，交易相关复杂交互和异步提交，借助奥创协议。
- 导购营销链路：动态化，高效，多端统一，主要选型是Weex，HTML5。
- 其他：端智能，动画偏Native实现，开放生态基于小程序。

注意：随着技术的成熟，这个也是演进变化的过程，例如手淘会从Weex向H5转。国际化复杂网络环境和应用市场审核风险，选型也会有所不同。

另外，组件往往与投放能力相关，对组件化技术的选择也绕不开搭投的选择，这个不做展开，毕竟现在没个搭投系统都不好意思出门，核心要考虑的是你造出的轮子（搭投）解决什么问题，受众是谁？是运营还是技术？是前端开发还是后端开发？组件化面临的很多问题和FAAS也是类似的，下一节展开。

▼ 国际化支付



内置 container

React

container

v1



内置 subContainer

React

subContainer

v1



内置 footer

React

footer

v1



内置 paymentHorizo...

React

paymentHorizontalTip

v1



内置 idAuthentication

React

idAuthentication

v1



内置 paymentCountd...

React

paymentCountdown

v1



内置 verifyCard

React

verifyCard

v1



内置 cardListManag...

React

cardListManagement

v1



内置 feedback

React

feedback

v1



ID	模版名	描述	作者	编译类型	正式版本	开发版本	更新时间	操作
5023	ae_home_promotion_tmall	AE首页，大促Tmall楼层	陈磊	3.5.0	8	9 开发中	01-03 11:59	属性 编辑
6612	ru_detail_one_row_entrance	Detail一行描述入口	陈磊	3.0.0	3		01-02 18:30	属性 编辑
6221	ae_home_flash_deal	AE首页，新版 Flash Deal	陈磊	3.2.0	14	15 开发中	2019-12-31	属性 编辑
6486	ae_russia_store_v2	Tmall Russia Store 2.0	陈磊	3.5.0	3		2019-12-30	属性 编辑
6247	ru_detail_review	AE detail 商品评价模块	陈磊	3.5.0	5	6 开发中	2019-12-30	属性 编辑
3894	ae_home_main_venue_entrance	AE首页，中促楼层	陈磊	3.2.0	11		2019-12-27	属性 编辑

图6 左侧为AStore搭建平台业务组件，右侧为DinamicX平台组件列表

FAAS化

FAAS和Serverless解决两个层面的问题，前者是**Function生命周期管理**，后者是运行时**弹性扩缩容**，目前FAAS的各种实现方案价值没有想象的大，只是**从后端工作转移到前端**（于前端的价值是不再仅仅和UI打交道，有了业务抓手），从IDE到WEB-IDE，从Java到JS，热部署的价值也被基于稳定性考虑而增加的诸多流程管控削弱。

SQL的价值带来大数据（ODPS）研发的大繁荣，极大降低了研发门槛。研发，测试，数据分析师，运营人人可参与，**业务创新成为可能**，同时table/task的linkage，meta系统也让资产积累和重用成为可能。类比到FAAS领域，**function发现**（能否基于星环的商业能力地图？）和**function治理**（图7是一个系统的公共function截图）也是选择FAAS平台必须考虑的问题，试想一个FAAS平台上有getItem，getItem2，getItem3ForXXX，对于function开发者如何选择，最后还是沦落为自己再写个getItem4ForYYY。

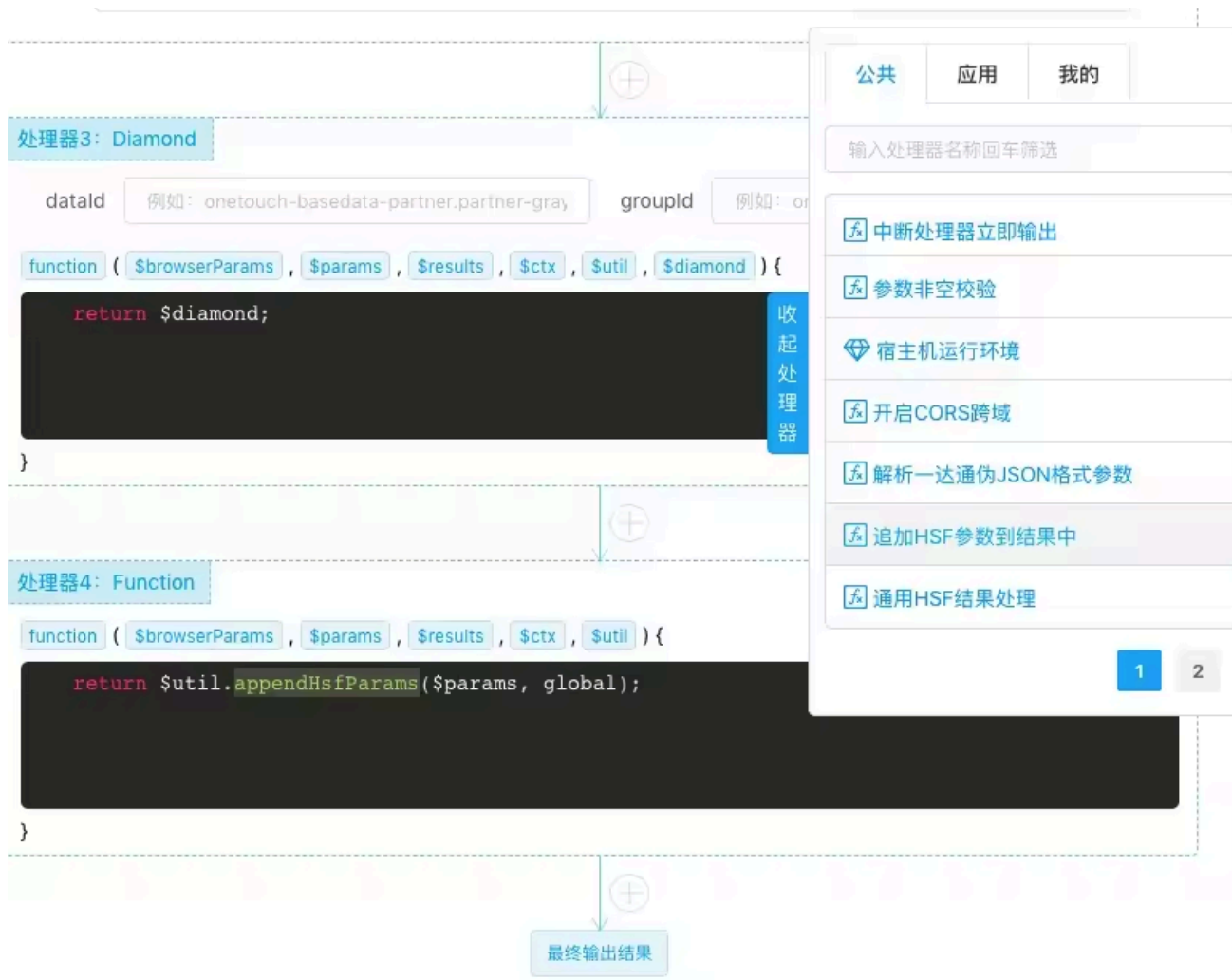


图7 陆游系统的通用Function

那么上面的问题怎么解决？有人提出了LCNC（Low Code or No Code），即低代码或无代码，正如前面对ODPS生态繁荣的分析，首先想到了SQL，业界有Facebook开源的GraphQL，集团内部也有TQL（如图8截图），背后有一套分布式的执行引擎，没去深入看过，我理解和hive/spark catalyst等类似是一套分布式执行计划，这个Plan复杂性是非常高的，你可以想想如何高效实现类似下面的语义：

```
select i.id, i.title, i.price, s.stock from Item i, Stock s where i.id=s.item_id limit 10;
```

延伸开来，这个QL是否要支持物化视图或临时表，例如下面：

```
insert into Payment.SavedCard (account, name, card_no)
```

```
select r.account, concat (r.first_name, r.last_name) , r.card_no from RequestContext r
```

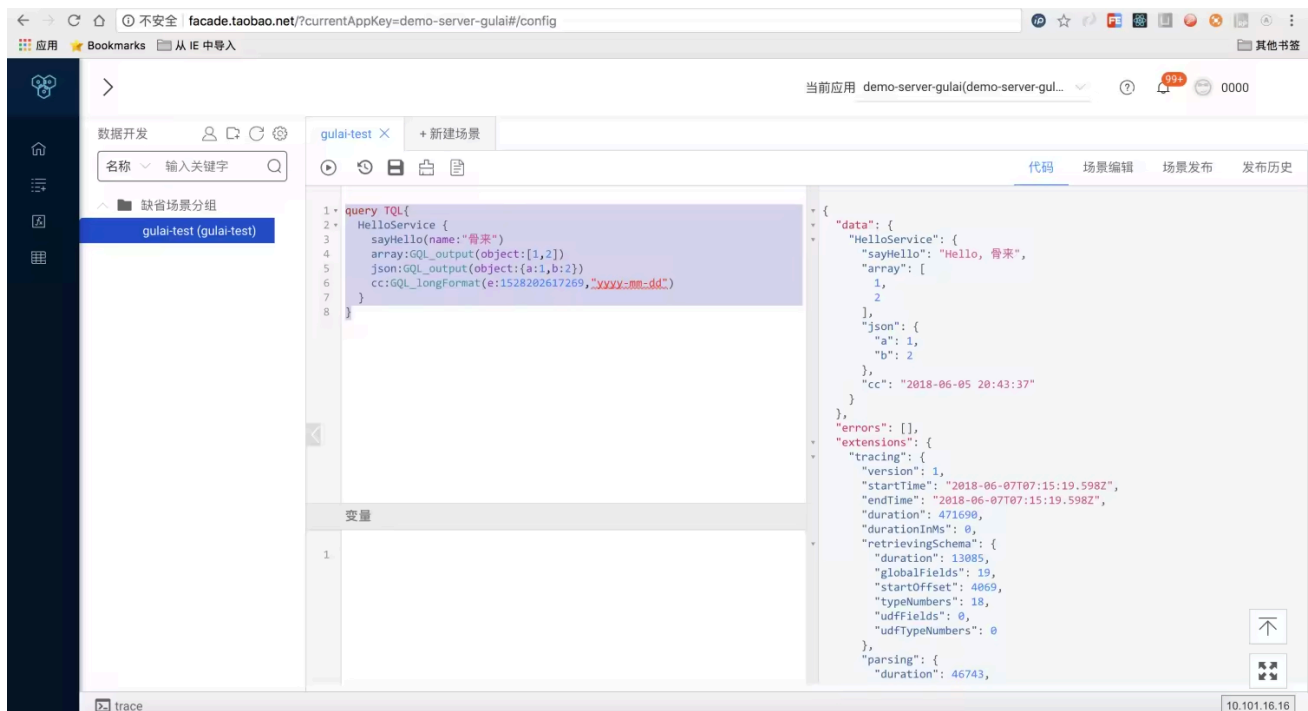


图8 TQL开发界面

还有另一个思路是定义适合自身业务特点的DSL，例如盒马的NBF，SOP就是他的DSL。

关于流程化编排，可以看两个平台的例子：

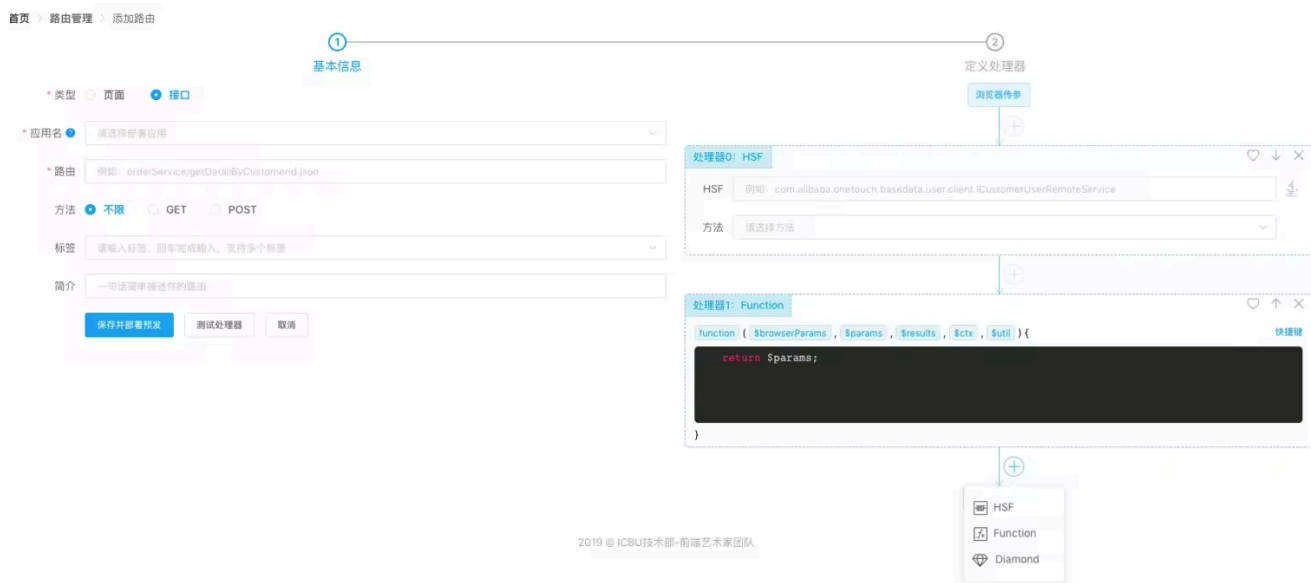


图9 陆游系统的处理器组装过程

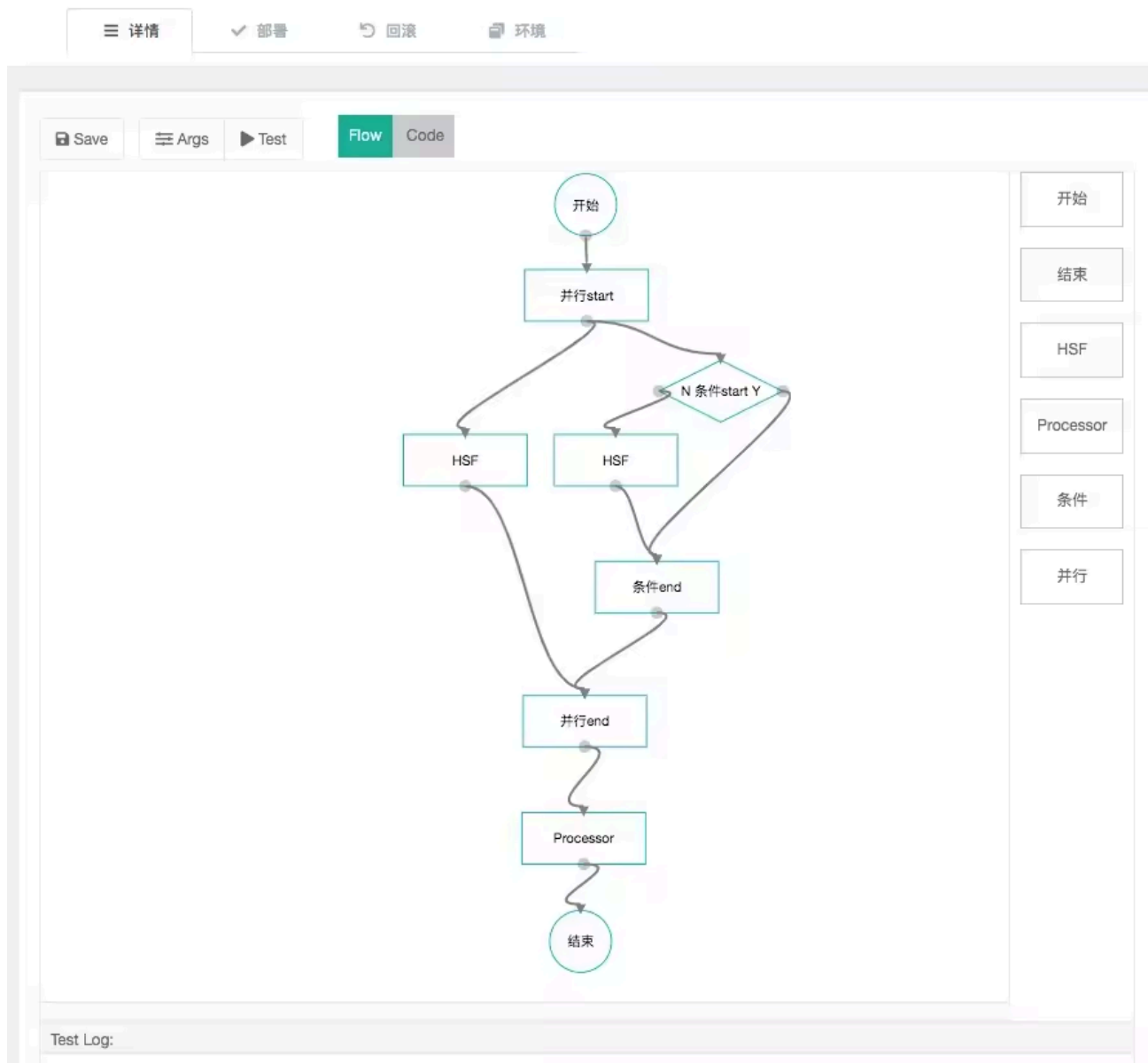


图10 阿基米德系统基于Camel实现的流程编排

从Serverless委员会大图可以看到最底层是弹性调度层，这块技术含量高，竞品就比较少，通过容器隔离提升function的稳定性，通过弹性调度和内存快照（或者checkpoint）实现毫秒级扩缩容，细节可以关注许晓斌团队的CSE，ATA的相关文章：[文章1](#)，[文章2](#)。另外原MTOP团队出品的[GAIA](#)（盖亚）也在做这块，并且融合了Service Mesh。

数据化

依据常识，我们的经验积累，认知升级遵从：发现问题》分析问题》解决问题》积累经验》完善自我 这样一个过程，我把数据化的过程也定义为：**目标》洞察》实验》优化** 这四个步骤。目前每个步骤都是散落在多个团队，多个产品，如果能在BFF层做成通用解决方案收益会比较大，目前ABTest是一个基础的工具，基于实验驱动的端优化（UI，数据，差异化等），但在AE首页的实践中也发现对于实验的解读是一个门槛比较高的事情，某个楼层的调

整如何作用于整个页面，从局部最优到全局最优。目前的思路是和BA同学一起定义每个场的核心衡量指标，建立基线，每一次业务调整对标基线的变化。

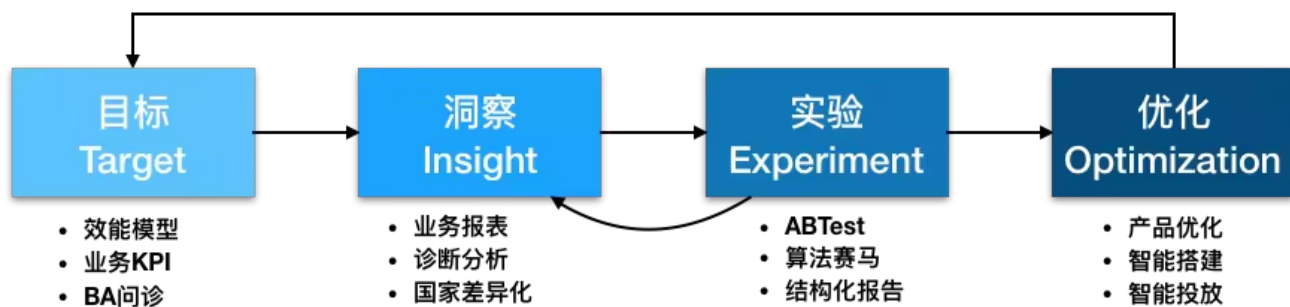


图11 一个美好的数据化链路

实践

在Sirius（AE下沉国际化中台）项目中，实践了支付域的BFF方案，逻辑架构和物理部署如下图所示。

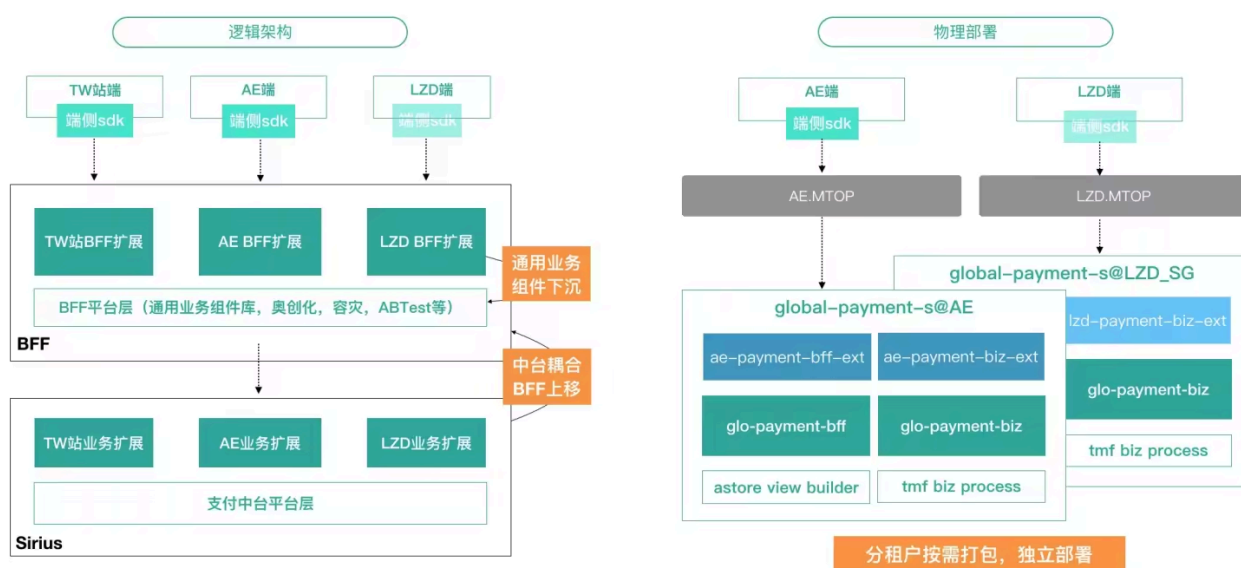


图12 BFF在国际化支付域的实践和规划

最后说一些感悟，想法很美好，但现实很残酷，BFF的范围界定，以及因此带来的生产关系变化，团队协作，谁是第一责任人；引入BFF带来的链路变长，问题排查监控等等问题，还需要大家继续探索前行。

本文的分析更多是从技术视角出发，但如何与业务结合，为业务带来价值，经过半年多的实践和讨论，meet at 全域投放，也是我目前在主要负责的业务领域，如果你感兴趣，可以私信聊聊。